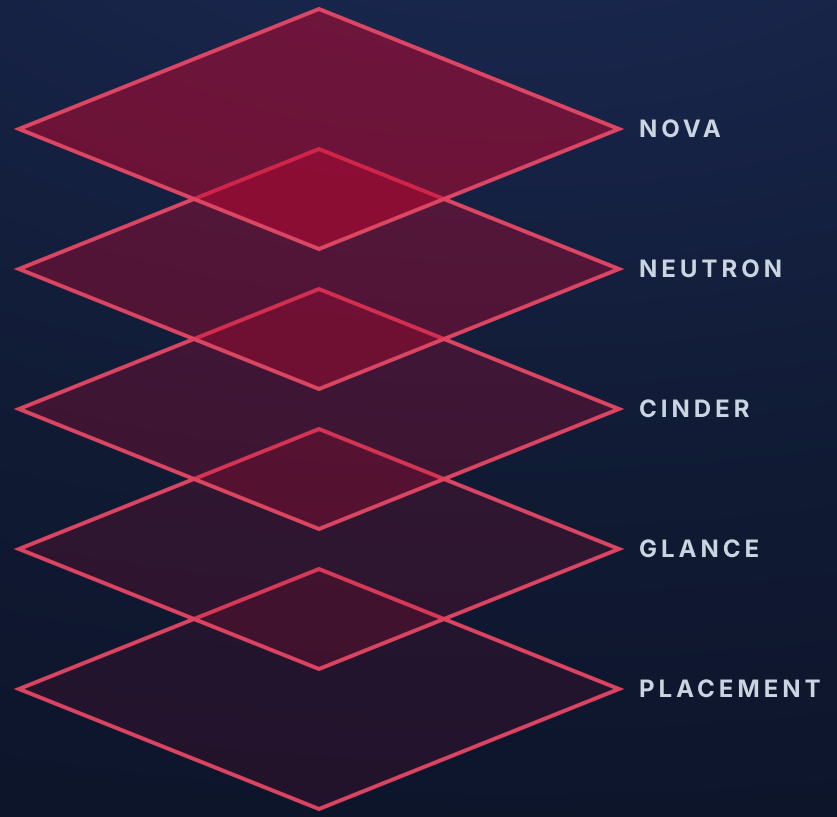
COMPRENDRE OPENSTACK · MODULE 4

Tour des principaux services

Images, gestion des ressources, calcul, stockage bloc, réseau : rôle, architecture, interactions

Glance | Placement | Nova | Cinder | Neutron

Version de référence : OpenStack 2026.2 « Hibiscus » (30 septembre 2026) · réseau de référence : ML2/OVN



INTRODUCTION

Objectif et déroulé

Objectif : comprendre comment chacun des cinq services centraux fonctionne de l'intérieur, et ce qu'il demande aux quatre autres pour accomplir sa tâche.

À l'issue de ce module, vous saurez :

- Décrire le **rôle** de Glance, Placement, Nova, Cinder et Neutron, et la frontière de chacun.
- Dessiner leur **architecture** : processus, pilotes, bases, backends.
- Lire le **cycle de vie** d'une image, d'une instance, d'un volume et d'un port à travers leurs états.
- Expliquer les **interactions** : qui appelle qui, à quel moment, pour obtenir quoi.
- Choisir entre les variantes de fonctionnement (téléchargement ou clone, disque local ou volume, routage distribué ou centralisé).
- Lancer les commandes qui montrent l'état de chaque service.

1 · Glance — images

Magasins, cycle de vie, import, propriétés qui pilotent les autres services.

2 · Placement — gestion des ressources

Fournisseurs, inventaires, allocations, caractéristiques, candidats.

3 · Nova — calcul

Cellules, états, ordonnancement, hyperviseur, disques, déplacements.

4 · Cinder — stockage bloc

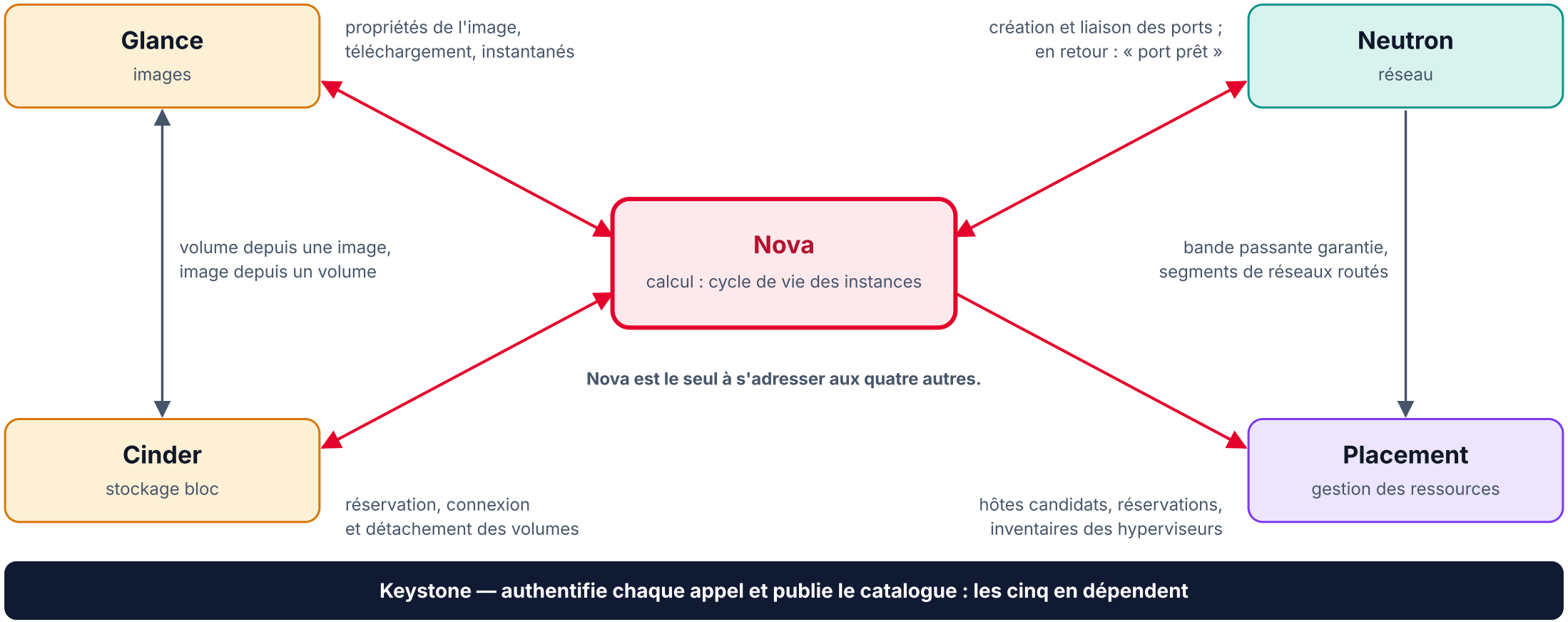
Backends, types, états, création depuis une image, sauvegardes.

5 · Neutron — réseau

ML2, traduction OVN, liaison de port, filtrage, chemins du trafic.

INTRODUCTION

Cinq services, et ce qu'ils se demandent



INTRODUCTION

Un même patron, cinq déclinaisons

Chaque service suit la même logique : une API qui valide et enregistre, un composant qui décide **où** réaliser la demande, un exécutant qui la réalise à travers un **pilote**, et un backend qui fait le travail réel.

	Glance	Placement	Nova	Cinder	Neutron
Ressource gérée	Image	Inventaire, allocation	Instance	Volume	Réseau, port, routeur
API	glance-api	placement-api	nova-api	cinder-api	neutron-server
Qui décide où ?	Magasin par défaut ou choisi	— (il informe la décision)	nova-scheduler	cinder-scheduler	Liaison de port (ML2)
Qui exécute ?	glance-api lui-même	—	nova-compute	cinder-volume	ovn-controller
Pilote	glance_store : rbd, file, swift…	—	libvirt, Ironic, VMware	RBD, LVM, baies	Mécanisme : ovn, openvswitch
Backend	Ceph, disque, objet	Sa base de données	KVM / QEMU	Ceph, LVM, baie	OVN + Open vSwitch
Bus de messages	Non	Non	Oui	Oui	Très peu avec OVN
Machine à états	Statut de l'image	—	vm_state, task_state	Statut du volume	Statut du port

Clé de lecture : pour comprendre un service, repérer ses quatre étages — API, décision, exécution, pilote — puis suivre une ressource d'un état à l'autre.

Placement est l'exception : il n'exécute rien. C'est un registre que les autres services alimentent et interrogent ; toute sa logique tient dans son modèle de données.

01

Glance — le service d'images

Glance tient le catalogue des images disque et les distribue aux services qui en ont besoin. Il ne lance rien, mais ce qu'il contient décide de la façon dont les instances démarrent.

- Rôle et notions
- Cycle de vie d'une image
- Propriétés qui pilotent Nova et Cinder
- Architecture : API, magasins, transit
- Import en deux temps et greffons
- Interactions, configuration, commandes

1 · GLANCE

Rôle et notions

Rôle : enregistrer, décrire, protéger et distribuer des **images disque**. Une image est le modèle à partir duquel on crée le disque système d'une instance ou un volume amorçable.

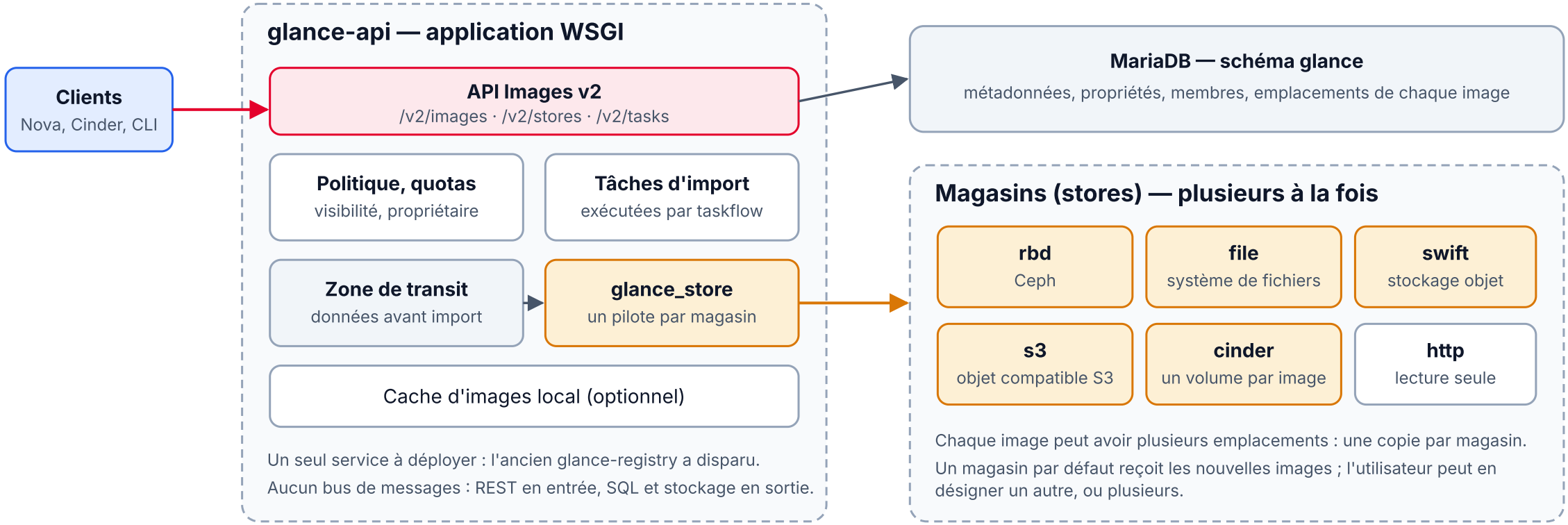
- **Ce qu'il fait :** conserver les données dans un ou plusieurs magasins, tenir les métadonnées en base, contrôler qui voit quoi.
- **Ce qu'il ne fait pas :** construire des images, les maintenir à jour, démarrer des machines.
- **Qui s'en sert :** Nova (démarrage, instantanés), Cinder (volumes amorçables), Ironic (déploiement de serveurs physiques), les utilisateurs.

Visibilité	Qui voit l'image
private	Le projet propriétaire
shared	Le propriétaire et les projets qu'il désigne comme membres
community	Tous, mais hors des listes par défaut
public	Tous ; réservé aux administrateurs

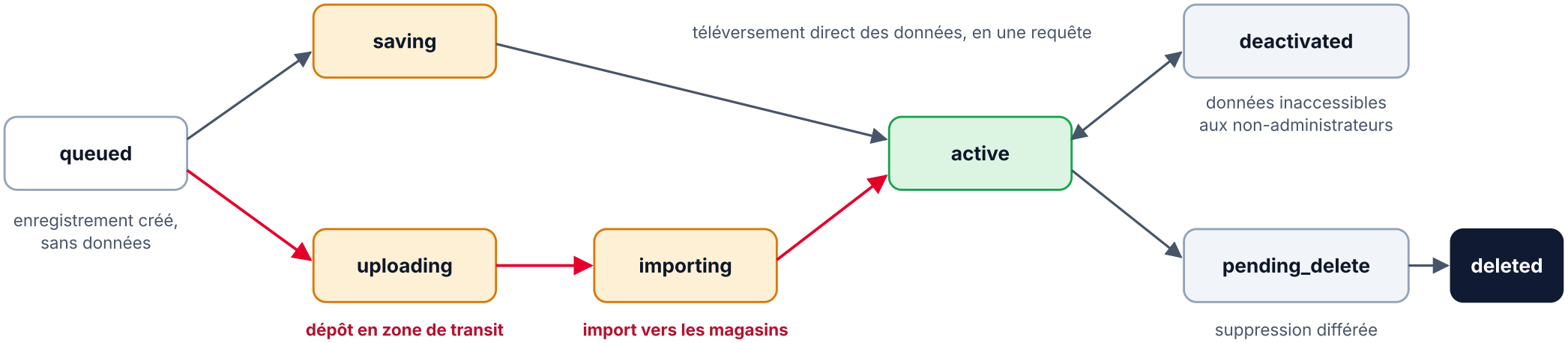
Notion	Définition
Format de disque	`qcow2` (compact), `raw` (brut, requis pour le clone Ceph), `vmdk`, `iso`…
Format de conteneur	Presque toujours `bare` : l'image n'est enveloppée dans rien
Somme de contrôle	Empreinte calculée à l'enregistrement, vérifiée par le consommateur
Taille minimale	`min_disk`, `min_ram` : refus des gabarits trop petits
Propriétés	Paires clé-valeur libres ; certaines sont lues par Nova et Cinder
Protection	`protected` : suppression interdite tant que le drapeau est posé
Emplacements	Une image peut exister dans plusieurs magasins à la fois
Instantané d'instance	Une image créée par Nova à partir d'une instance

Une image est immuable : une fois active, ses données ne changent plus. Mettre à jour un système, c'est publier une **nouvelle** image et retirer l'ancienne.

Architecture : une API, une base, des magasins



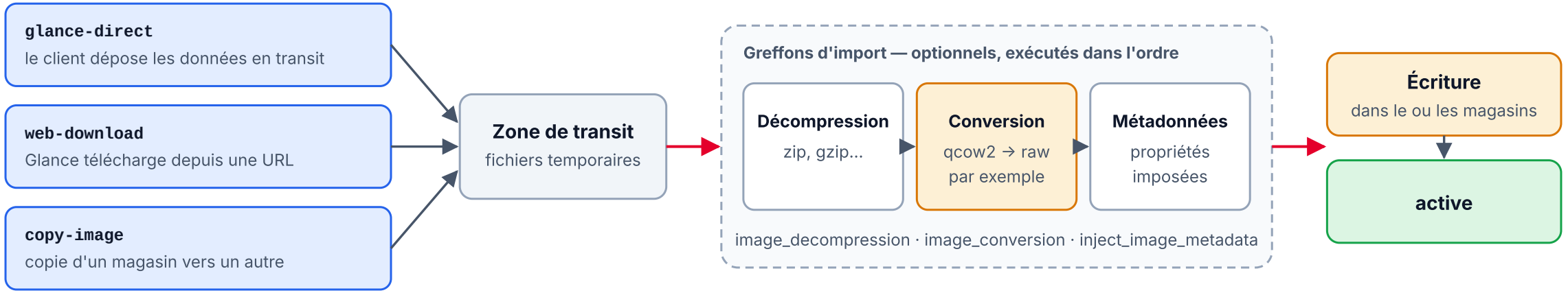
Cycle de vie d'une image



Statut	Signification
queued	L'image existe en base, ses données ne sont pas encore fournies
saving	Données en cours de réception, par l'ancien mode de téléversement direct
uploading, importing	Données en transit, puis en cours de traitement et d'écriture dans les magasins
active	Utilisable : somme de contrôle calculée, au moins un emplacement

- Seule une image **active** peut servir à démarrer une instance ou créer un volume.
- Un import en échec laisse en général l'image à l'état `importing` ou la ramène à `queued` : on corrige, puis on relance.
- Supprimer une image n'affecte pas les instances déjà créées ; si leurs disques en sont des clones Ceph, Glance refuse la suppression.

L'import en deux temps et ses greffons




```
# 1. Créer l'enregistrement
openstack image create ubuntu-24.04 --disk-format qcow2 \
    --container-format bare
# 2. Déposer puis importer (client glance)
glance image-stage --file noble.qcow2 <id>
glance image-import --import-method glance-direct <id>
# Ou tout en une commande
glance image-create-via-import --name ubuntu-24.04 \
    --disk-format qcow2 --container-format bare --file noble.qcow2
```

- **Pourquoi deux temps :** l'opérateur peut contrôler ou transformer l'image avant qu'elle ne devienne utilisable.
- **Conversion vers raw :** indispensable pour profiter du clone instantané avec Ceph, quel que soit le format fourni par l'utilisateur.
- Avec plusieurs contrôleurs, la zone de transit doit être **partagée**, ou chaque glance-api doit pouvoir joindre les autres.
- Une quatrième méthode, `glance-download`, importe depuis le Glance d'une autre région.

> Méthodes proposées aux utilisateurs : option `enabled_import_methods` ; par défaut `glance-direct` et `web-download`.

1 · GLANCE

Les propriétés d'image qui pilotent les autres services

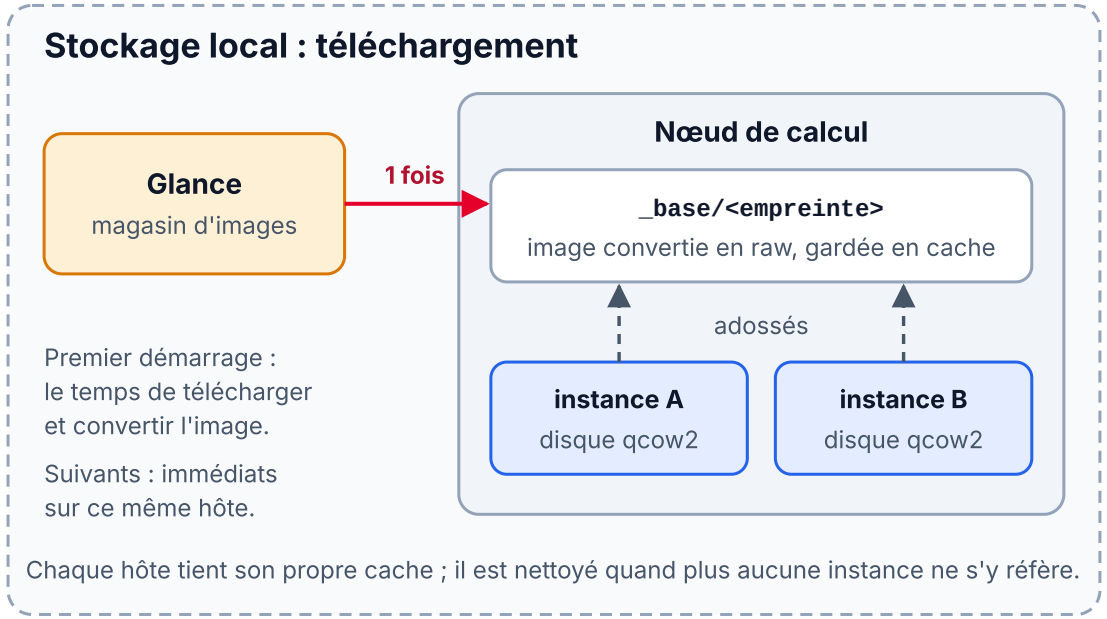
Glance ne les interprète pas : il les conserve. Ce sont Nova et Cinder qui les lisent au moment de créer l'instance ou le volume.

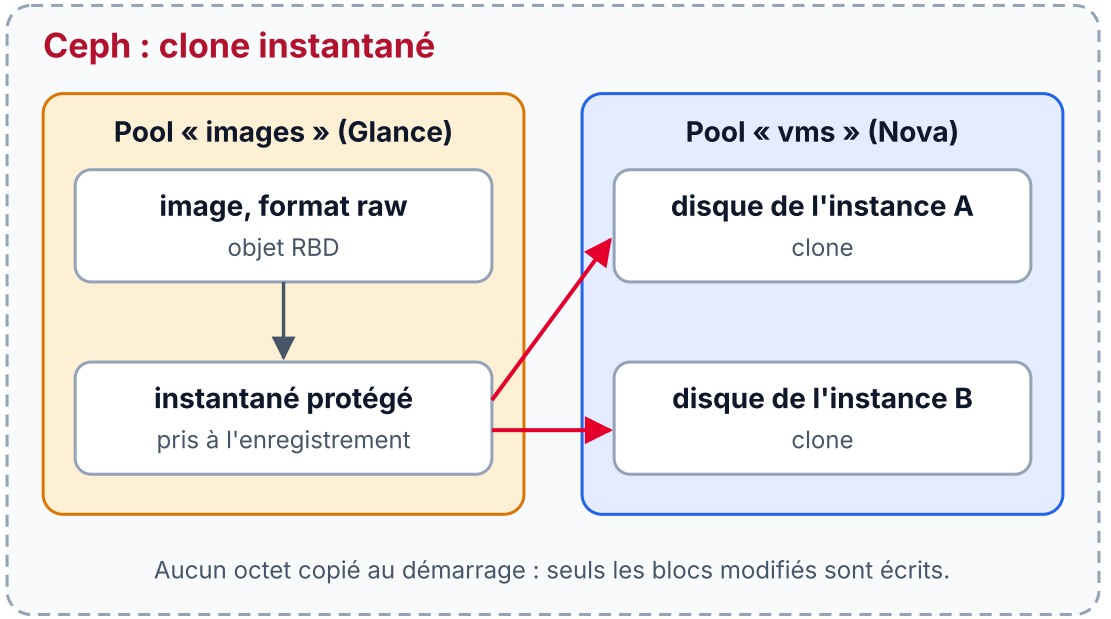
Propriété	Lue par	Effet	Exemple
`min_disk` , `min_ram`	Nova (API)	Refuse un gabarit plus petit que l'image ne l'exige	20 Go, 2048 Mo
`hw_disk_bus` , `hw_scsi_model`	Nova (libvirt)	Type de contrôleur disque présenté à la machine	`scsi` , `virtio-scsi`
`hw_vif_model`	Nova (libvirt)	Modèle de carte réseau virtuelle	`virtio` , `e1000`
`hw_firmware_type` , `hw_machine_type`	Nova (libvirt)	BIOS ou UEFI ; type de carte mère émulée	`uefi` , `q35`
`os_type` , `os_distro`	Nova	Réglages adaptés au système invité (horloge, format du disque éphémère)	`linux` , `windows`
`hw_qemu_guest_agent`	Nova	Active le canal de l'agent invité : gel des systèmes de fichiers avant instantané	`yes`
`hw_cpu_policy` , `hw_numa_nodes`	Nova (ordonnancement)	CPU dédiés, topologie NUMA exigée	`dedicated`
`trait:…=required`	Nova → Placement	N'accepte que les hôtes portant cette caractéristique	`trait:HW_CPU_X86_AVX2`
`img_signature_*`	Nova, Cinder	Vérification de la signature de l'image avant usage	Certificat dans Barbican
`cinder_img_volume_type`	Cinder	Type de volume par défaut pour un volume créé depuis cette image	`ssd`

Gabarit et image se complètent : la plupart de ces réglages existent aussi comme propriétés de gabarit (`hw:cpu_policy`). En cas de contradiction, Nova refuse la demande ou donne la priorité au gabarit, choisi par l'opérateur.

Conséquence d'exploitation : une image publiée sans ses propriétés (UEFI, bus disque) peut démarrer… ou rester bloquée au démarrage sans message. Elles font partie de l'image au même titre que ses données.

Interaction avec Nova : télécharger ou cloner





Condition du clone Ceph	Réglage
Image au format raw	Conversion à l'import, côté Glance
Même cluster Ceph	Glance et Nova sur des pools du même cluster
Emplacement visible de Nova	Glance : `show_image_direct_url` ; Nova : `images_type = rbd`

- Si une condition manque, Nova **retombe sans prévenir** sur le téléchargement complet : démarrages lents, espace consommé en double.
- Dans les deux cas, Nova vérifie la somme de contrôle de ce qu'il reçoit.
- **Instantané d'instance :** chemin inverse — Nova fige le disque et le téléverse dans Glance comme nouvelle image ; avec Ceph, c'est encore un clone.

Les autres interactions de Glance

Avec	Sens	Ce qui se passe	Point d'attention
Keystone	Glance → Keystone	Validation du jeton ; le projet du jeton détermine les images visibles	Une image « introuvable » appartient souvent à un autre projet
Nova	Nova → Glance	Lecture des propriétés, téléchargement ou clone, téléversement des instantanés	Nova garde l'identifiant de l'image dans l'instance, pas ses données
Cinder	Cinder → Glance	Création d'un volume amorçable depuis une image ; export d'un volume en image	Clone direct possible si image et volumes partagent le même cluster Ceph
Cinder (magasin)	Glance → Cinder	Avec le magasin `cinder`, chaque image est stockée dans un volume	Utile quand le seul stockage disponible est une baie pilotée par Cinder
Swift, Ceph, S3	Glance → stockage	Lecture et écriture des données d'image par le pilote du magasin	La disponibilité de Glance est celle de son magasin
Barbican	Nova, Cinder → Barbican	Récupération du certificat pour vérifier la signature d'une image	Glance ne fait que conserver les propriétés de signature
Ironic	Ironic → Glance	Récupération de l'image à écrire sur le disque d'un serveur physique	Images souvent au format raw, parfois volumineuses

Glance n'appelle presque personne : il est sollicité. Sa seule dépendance active est son magasin de données — et Keystone, comme tout le monde.

Sécurité des formats : une image qcow2 peut référencer un fichier externe. Glance et Nova inspectent le format réel des images et refusent celles qui ne correspondent pas au format déclaré ; restreindre `disk_formats` à ce qui est réellement utilisé.

Configuration type et commandes

```
# /etc/glance/glance-api.conf — deux magasins
[DEFAULT]
enabled_backends = ceph:rbd, local:file
enabled_import_methods = glance-direct,web-download,copy-image
show_image_direct_url = true

[glance_store]
default_backend = ceph

[ceph]
rbd_store_pool = images
rbd_store_user = glance
rbd_store_ceph_conf = /etc/ceph/ceph.conf

[local]
filesystem_store_datadir = /var/lib/glance/images/

[image_import_opts]
image_import_plugins = ['image_conversion']
[image_conversion]
output_format = raw
```

```
# Magasins déclarés
glance stores-info

# Publier une image avec ses propriétés
openstack image set ubuntu-24.04 --public --protected \
    --min-disk 10 --property hw_firmware_type=uefi \
    --property hw_machine_type=q35 --property os_distro=ubuntu

# État, format, somme de contrôle, emplacements
openstack image show ubuntu-24.04 -c status -c disk_format \
    -c checksum -c properties

# Copier vers un second magasin ; partager avec un projet
glance image-import <id> --import-method copy-image --stores local
openstack image add project <image> <projet>
```

- `enabled_backends` associe un nom libre à un pilote ; chaque nom a sa section.
- `show_image_direct_url` expose l'emplacement réel : nécessaire au clone, mais à réserver à un Glance joint par l'interface interne.

02

Placement — la gestion des ressources

Placement tient la comptabilité du cloud : ce que chaque fournisseur possède, ce qui en est déjà consommé, et par qui. C'est lui qui permet de répondre à la question « où reste-t-il de la place ? ».

- Rôle et frontière avec Nova
- Calcul de la capacité
- La requête de candidats
- Modèle de données
- Arbres et fournisseurs partagés
- Interactions, commandes, diagnostic

Rôle : un registre, pas un ordonnanceur

Rôle : enregistrer les **quantités** de ressources disponibles et consommées, et répondre aux requêtes du type « quels fournisseurs peuvent offrir 2 VCPU, 4 Go de mémoire et 20 Go de disque, avec telle caractéristique ? ».

- **Ce qu'il fait :** tenir inventaires et allocations, filtrer par quantité, caractéristique et agrégat, garantir qu'une ressource n'est pas promise deux fois.
- **Ce qu'il ne fait pas :** choisir l'hôte final, connaître les instances, parler aux hyperviseurs. Il ne sait pas ce qu'est une machine virtuelle.
- **Né dans Nova**, il en a été extrait pour servir aussi Neutron, Cyborg, Ironic et d'autres.

Partage des tâches : Placement réduit des milliers d'hôtes à une courte liste de candidats **possibles** ; nova-scheduler choisit ensuite le **meilleur** selon des critères que Placement ignore (affinité, charge, politique).

Question	Qui répond
Cet hôte a-t-il assez de VCPU, de mémoire, de disque ?	Placement
Porte-t-il la caractéristique exigée (AVX2, SSD) ?	Placement
Est-il dans la zone de disponibilité demandée ?	Placement (agrégats)
Est-il désactivé par l'administrateur ?	Placement (caractéristique dédiée)
Respecte-t-il l'anti-affinité du groupe de serveurs ?	nova-scheduler (filtre)
Sa topologie NUMA convient-elle ?	nova-scheduler (filtre)
Lequel des candidats est le moins chargé ?	nova-scheduler (pondération)

- Un seul processus : **placement-api**, une application WSGI et sa base. Ni bus de messages, ni agent.
- Sans état propre en mémoire : tout est en base, chaque requête est indépendante.
- Indisponible, il bloque toute création et tout déplacement d'instance ; les instances existantes ne sont pas touchées.

Le modèle de données



Quantités d'un côté, qualités de l'autre : les **classes de ressources** se comptent et se consomment (VCPU, MEMORY_MB, DISK_GB, VGPU, SRIOV_NET_VF) ; les **caractéristiques** sont présentes ou absentes et ne s'épuisent pas. Une requête combine toujours les deux.

Le calcul de la capacité

VCPU sur cmp01 — capacité = (total − réservé) × ratio = (64 − 0) × 4,0 = 256



MEMORY_MB sur cmp01 — capacité = (262 144 − 8 192) × 1,0 = 253 952

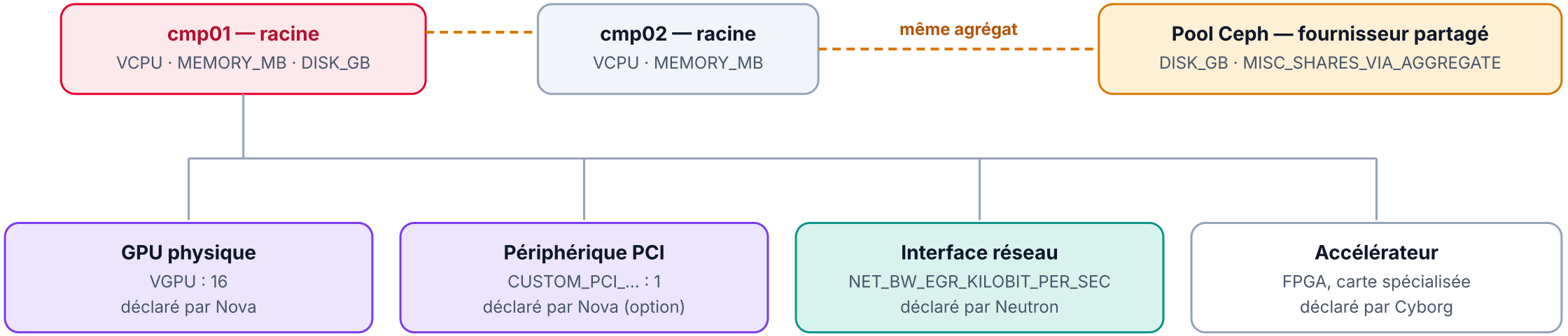


Champ d'un inventaire	Rôle
`total`	Quantité physique déclarée par le fournisseur
`reserved`	Part gardée pour l'hôte lui-même, jamais attribuée
`allocation_ratio`	Facteur de surallocation ; 1,0 = aucune
`min_unit` , `max_unit`	Plus petite et plus grande quantité qu'une seule allocation peut prendre
`step_size`	Pas : les quantités demandées en sont des multiples

- Une demande est acceptée si **alloué + demandé ≤ capacité**, pour chaque classe, et si elle respecte `max_unit` .
- Avec 76 VCPU libres mais `max_unit = 64` , une instance de 72 VCPU est refusée : elle ne tiendrait pas sur les cœurs réels.
- Les ratios viennent de `nova.conf` sur chaque hyperviseur (`cpu_allocation_ratio` …) ou sont fixés par l'API de Placement.
- La mémoire se suralloue rarement : au-delà de 1,0, l'hôte risque de tuer des machines faute de mémoire.

> Ce sont des **promesses**, pas des mesures : Placement compte ce qui est réservé par les gabarits, pas ce que les instances consomment réellement.

Arbres de fournisseurs et ressources partagées



Une même requête peut être servie par plusieurs fournisseurs d'un arbre : VCPU et mémoire sur la racine, VGPU sur un enfant, disque sur le pool partagé.

Fournisseurs imbriqués

Un hyperviseur n'est pas un bloc uniforme : un GPU, une carte SR-IOV ont leur propre inventaire et leurs propres caractéristiques. Ils sont déclarés comme enfants du nœud de calcul.

Fournisseurs partagés

Un pool de stockage commun à plusieurs hôtes n'est compté qu'**une fois**. Il est rattaché à ces hôtes par un agrégat ; sans cela, chaque hôte déclarerait le pool entier comme sien.

Plusieurs déclarants

Nova n'est pas seul à écrire : Neutron et Cyborg ajoutent leurs fournisseurs sous l'arbre du même hôte. Une caractéristique récente, `OWNER_NOVA`, distingue ceux que Nova gère.

La requête centrale : les candidats à l'allocation

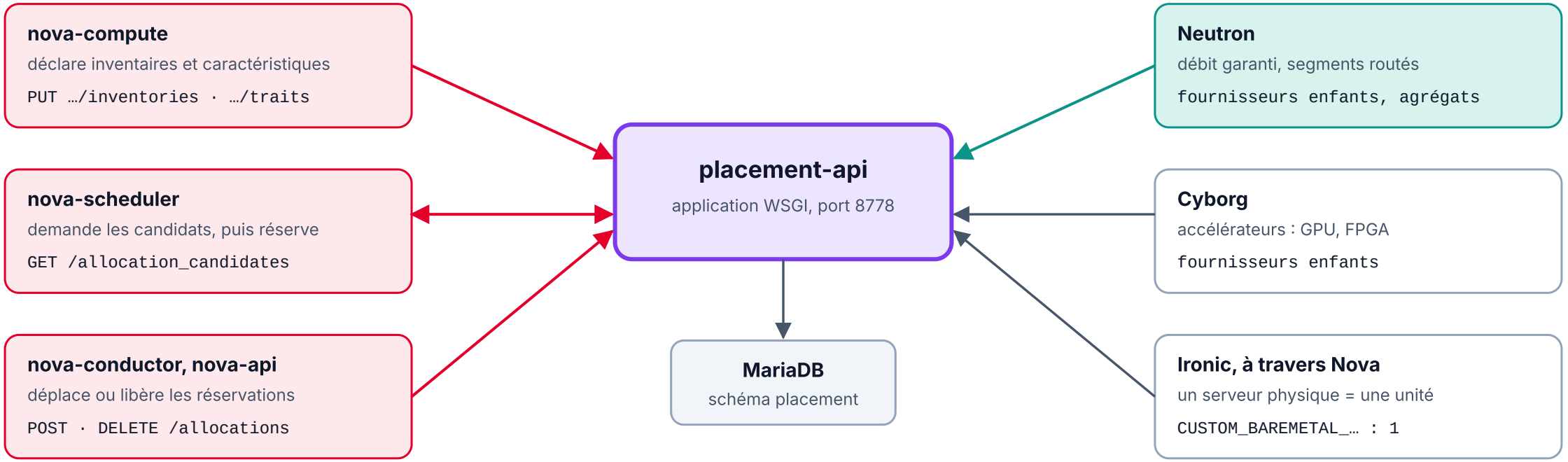
```
GET /allocation_candidates
    ?resources=VCPU:2,MEMORY_MB:4096,DISK_GB:20
    &required=HW_CPU_X86_AVX2,!COMPUTE_STATUS_DISABLED
    &member_of=in:3f2a…          # agrégat de la zone az-nord
    &limit=1000

# Réponse (simplifiée)
{
  "allocation_requests": [
    { "allocations": {
        "4e8a… (cmp01)": { "resources":
            { "VCPU": 2, "MEMORY_MB": 4096, "DISK_GB": 20 } } } },
    { "allocations": { "77b1… (cmp07)": { … } } }
  ],
  "provider_summaries": {
    "4e8a…": { "resources": { "VCPU":
        { "capacity": 256, "used": 180 }, … },
              "traits": [ "HW_CPU_X86_AVX2", … ] }
  }
}
```

Paramètre	Sens
`resources`	Quantités exigées, par classe
`required`	Caractéristiques exigées ; `!` pour une caractéristique interdite
`member_of`	Restreint aux fournisseurs d'un ou plusieurs agrégats
`in_tree`	Restreint à l'arbre d'un fournisseur donné (hôte imposé)
`resourcesN` , `requiredN`	Groupes numérotés : ces ressources doivent venir d'un même fournisseur
`group_policy`	`isolate` : les groupes doivent venir de fournisseurs distincts
`limit`	Nombre maximal de candidats renvoyés

- **allocation_requests :** des réservations prêtes à l'emploi ; le demandeur en renvoie une telle quelle pour réserver.
- **provider_summaries :** l'état de chaque fournisseur cité, pour aider à choisir.
- La réservation (`PUT /allocations/{consommateur}`) est **atomique** : si un concurrent a pris la place entre-temps, elle échoue et le demandeur essaie le candidat suivant.

Architecture et interactions



Moment	Qui appelle Placement	Effet
Démarrage puis toutes les minutes	nova-compute (suivi des ressources)	Crée ou met à jour son fournisseur, ses inventaires et ses caractéristiques
Création d'une instance	nova-scheduler	Obtient les candidats, réserve sur l'hôte retenu avant même de le prévenir
Migration, redimensionnement	nova-conductor	Réserve sur la cible ; la source reste réservée jusqu'à confirmation
Suppression	nova-compute, nova-api	Libère les allocations du consommateur
Port à débit minimal garanti	Neutron déclare, Nova réserve	La bande passante devient une ressource comptée comme les autres

Commandes et diagnostic

```
# Fournisseurs, puis détail de l'un d'eux
openstack resource provider list
openstack resource provider inventory list <uuid>
openstack resource provider usage show <uuid>
openstack resource provider trait list <uuid>
openstack resource provider aggregate list <uuid>

# Qui consomme sur cet hôte ?
openstack resource provider show <uuid> --allocations

# Ce qu'une instance a réservé, et où
openstack resource provider allocation show <instance>

# Rejouer la question du scheduler
openstack allocation candidate list \
    --resource VCPU=2 --resource MEMORY_MB=4096 \
    --resource DISK_GB=20 --required HW_CPU_X86_AVX2

# Réparer les écarts entre Nova et Placement
nova-manage placement audit --verbose
nova-manage placement heal_allocations
```

Symptôme	Cause fréquente
« No valid host was found » alors que des hôtes semblent libres	Capacité épuisée dans **une** classe (souvent DISK_GB ou MEMORY_MB) ; caractéristique exigée absente
Aucun candidat renvoyé	Zone de disponibilité sans hôte éligible ; hôtes tous désactivés
Hôte plein dans Placement, vide en réalité	Allocations orphelines après une suppression ou une migration interrompue
Nouvel hyperviseur jamais choisi	Hôte non rattaché à une cellule (`discover_hosts`) ; inventaire non déclaré
Conflit « generation »	Deux écritures concurrentes sur un même fournisseur : le client doit relire et réessayer

Méthode devant un « No valid host » : rejouer la demande avec `allocation candidate list`. Aucun candidat : le problème est dans Placement (quantité, caractéristique, agrégat). Des candidats existent : ce sont les filtres de nova-scheduler qui les ont tous écartés.

Ces commandes viennent de l'extension `osc-placement` du client OpenStack.

03

Nova — le service de calcul

Nova gère le cycle de vie des instances. C'est le chef d'orchestre : il ne possède ni les images, ni les réseaux, ni les volumes, mais il les assemble autour d'une machine virtuelle.

- Rôle et processus
- Les états d'une instance
- nova-compute, pilote et disques
- Cellules : l'architecture à deux niveaux
- Ordonnancement : filtres et pondérations
- Déplacements, interactions, diagnostic

Rôle et processus

Rôle : créer, démarrer, arrêter, déplacer et supprimer des **instances** — machines virtuelles le plus souvent, serveurs physiques avec Ironic.

- **Ce qu'il possède :** les instances, les gabarits, les paires de clés, les groupes de serveurs, les agrégats d'hôtes.
- **Ce qu'il emprunte :** l'image (Glance), les ports (Neutron), les volumes (Cinder), la place (Placement).
- **Ce qu'il ne fait pas :** surveiller ce qui tourne dans l'instance, garantir sa disponibilité, sauvegarder ses données.

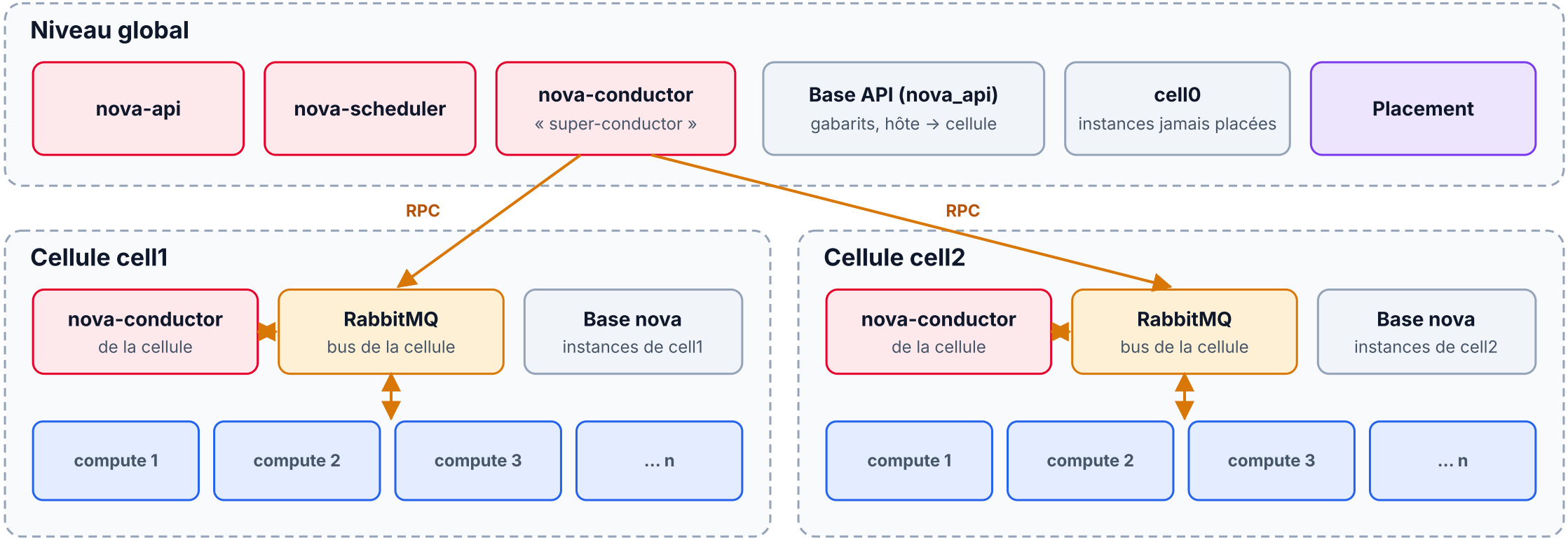
Notion	Définition
Gabarit (flavor)	Quantités de ressources et réglages matériels (`hw:…`)
Groupe de serveurs	Politique d'affinité ou d'anti-affinité, stricte ou souple
Agrégat d'hôtes	Regroupement d'hyperviseurs par l'administrateur, avec métadonnées
Zone de disponibilité	Un agrégat rendu visible des utilisateurs
Correspondance de périphériques bloc	Liste des disques de l'instance : image, volume, éphémère, swap

Processus	Où	Fonction
nova-api	Contrôle	API REST ; valide, vérifie les quotas, enregistre, délègue
nova-scheduler	Contrôle	Choisit l'hôte : Placement, filtres, pondérations
nova-conductor	Contrôle	Orchestre les opérations longues ; accès base pour les hôtes
nova-compute	Chaque hyperviseur	Exécute : pilote libvirt, disques, interfaces, volumes
API de métadonnées	Contrôle	Sert `169.254.169.254` : données utilisateur, clés
nova-novncproxy	Contrôle	Relaie les consoles vers le navigateur

Deux bases, pas une : la base « API » (`nova_api`) pour ce qui est global — gabarits, quotas, cartographie des cellules ; une base par **cellule** (`nova`) pour les instances et leurs hôtes.

Les quotas peuvent être comptés à partir de Placement et définis dans Keystone (« limites unifiées ») : Nova n'a alors plus de table de quotas propre.

Cellules : une architecture à deux niveaux



Pourquoi

Une base et un bus uniques finissent par saturer. Chaque cellule isole les siens : la charge et les pannes restent locales.

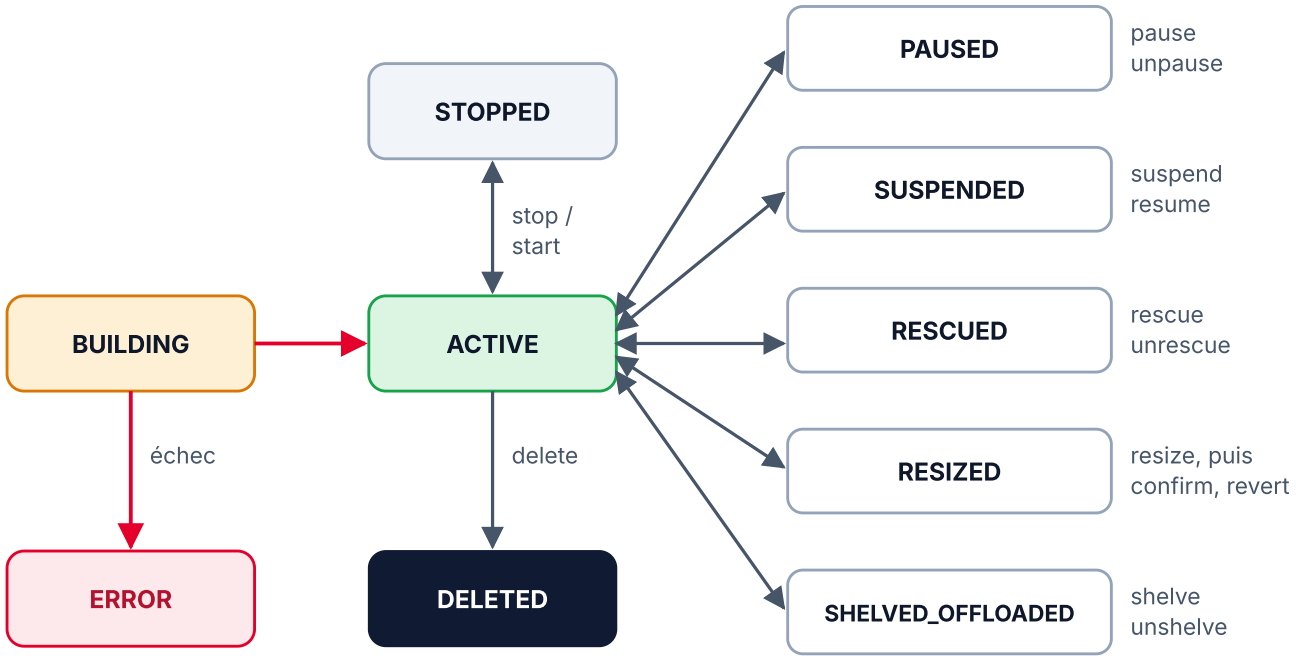
Comment

La base API sait dans quelle cellule vit chaque instance et chaque hôte. L'API s'adresse ensuite directement à la base et au bus de cette cellule.

À savoir

Tout déploiement a au moins `cell0` et `cell1`. Un nouvel hyperviseur doit être rattaché : `nova-manage cell_v2 discover_hosts`.

Les états d'une instance



ERROR est atteignable depuis presque tous les états, à la suite d'une opération en échec.

Champ	Ce qu'il décrit
`vm_state`	État stable de l'instance, du point de vue de Nova
`task_state`	Opération en cours ; vide quand rien ne se passe
`power_state`	Ce que l'hyperviseur constate réellement
`status`	Synthèse des deux premiers, montrée par l'API

- Pendant une création, `task_state` passe par `scheduling`, `networking`, `block_device_mapping`, `spawning` : il dit **à quelle étape** l'instance est restée bloquée.
- Une instance en cours d'opération refuse toute autre action : erreur 409 tant que `task_state` n'est pas vide.
- **SHELVED_OFFLOADED :** disque sauvegardé dans Glance, hôte et ressources libérés ; au retour, l'instance est replacée.
- Si `power_state` contredit `vm_state`, une tâche périodique réaligne la base sur la réalité.

Remettre une instance en état : `openstack server set --state active` corrige la **base**, pas l'hyperviseur. À n'utiliser qu'après avoir vérifié la réalité.

Ordonnancement : préfiltres, filtres, pondérations

1 · Préfiltres (via Placement)

- **Quantités** du gabarit : VCPU, mémoire, disque
- **Caractéristiques** exigées par le gabarit ou l'image
- **Zone de disponibilité** demandée → agrégat
- **Hôtes désactivés** exclus (`COMPUTE_STATUS_DISABLED`)
- **Format d'image** pris en charge par l'hôte (option)
- **Isolation** de projets ou d'agrégats (option)

> Tout ce qui s'exprime en quantité, caractéristique ou agrégat est traité **dans la base de Placement**, en une requête.

2 · Filtres (nova-scheduler)

- `ComputeFilter` : service actif
- `ImagePropertiesFilter` : architecture, hyperviseur
- `ComputeCapabilitiesFilter` : capacités exigées par le gabarit
- `ServerGroupAntiAffinityFilter` , `ServerGroupAffinityFilter`
- `NUMATopologyFilter` : CPU dédiés, pages larges
- `PciPassthroughFilter` : périphériques attribués
- `AggregateInstanceExtraSpecsFilter` : gabarit ↔ agrégat

> Un filtre répond oui ou non, hôte par hôte. Liste active : `[filter_scheduler] enabled_filters` .

3 · Pondérations

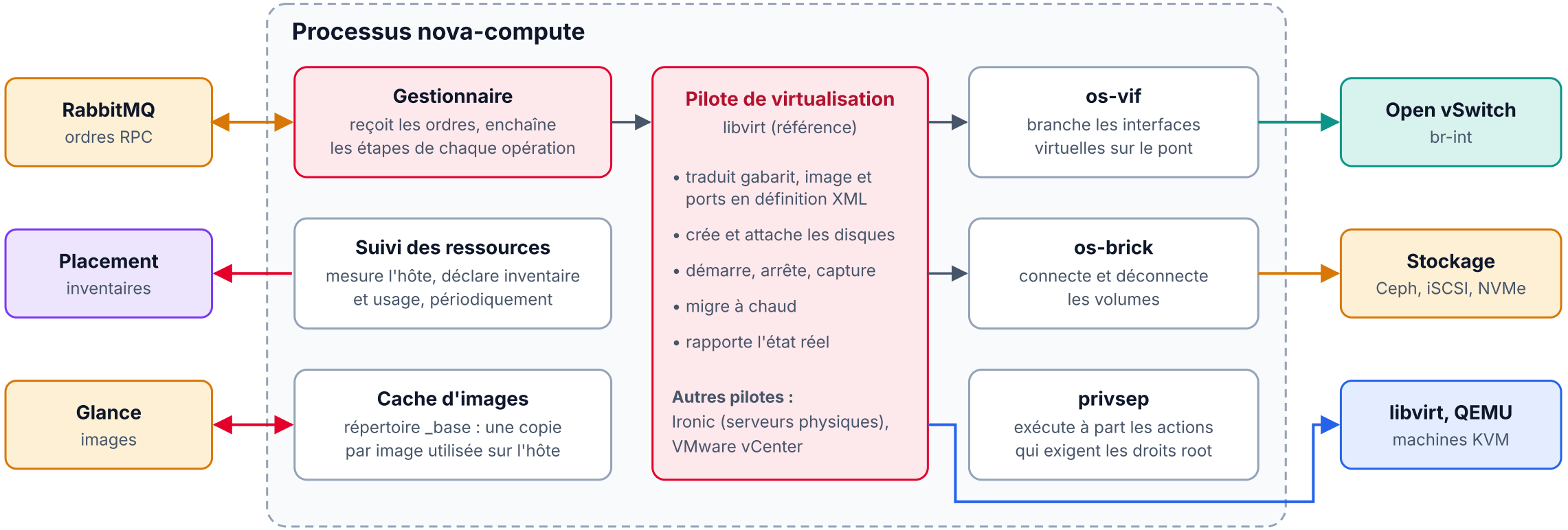
- `RAMWeigher` , `CPUWeigher` , `DiskWeigher` : ressources libres
- `IoOpsWeigher` : opérations en cours sur l'hôte
- `BuildFailureWeigher` : échecs récents
- `ServerGroupSoft…Weigher` : affinité souple
- `NumInstancesWeigher` , `PCIWeigher` …

> Chaque pondération note les hôtes restants ; un multiplicateur fixe son poids. Positif : **répartir**. Négatif : **tasser**.

Après le classement : le premier hôte est retenu, sa place réservée dans Placement, et quelques **suppléants** sont transmis avec la demande. Si la construction échoue sur l'hôte, le conductor réessaie sur le suivant, sans repasser par le scheduler (`max_attempts`).

Lire un refus : en mode `debug` , nova-scheduler journalise, filtre par filtre, combien d'hôtes restent. Le filtre qui fait tomber le compte à zéro désigne la contrainte impossible à satisfaire.

nova-compute : ce qui se passe sur l'hyperviseur



Un seul nova-compute par hyperviseur, et c'est lui qui « possède » les instances de cet hôte : s'il s'arrête, elles continuent de tourner mais plus aucune opération sur elles n'est possible.

Où vivent les disques d'une instance



	Fichier local	Ceph RBD	Démarrage sur volume
Réglage	`[libvirt] images_type = qcow2`	`images_type = rbd`, `images_rbd_pool = vms`	Demandé à la création de l'instance
Création	Rapide si l'image est déjà en cache	Instantanée (clone)	Selon le backend : clone ou copie
Migration à chaud	Avec copie des disques entre hôtes	Oui, sans copie	Oui, sans copie
Perte de l'hyperviseur	Disque perdu avec l'hôte	Instance relançable ailleurs	Instance relançable ailleurs
Suppression de l'instance	Disque supprimé	Disque supprimé	Volume conservé si demandé
Instantané	Téléversé dans Glance	Clone dans Glance	Instantané de volume, dans Cinder

> Dans tous les cas, une instance peut recevoir en plus un **disque de configuration** (données de démarrage en lecture seule), un disque éphémère supplémentaire et un disque d'échange, selon le gabarit.

Déplacer une instance : cinq opérations, cinq usages

Opération	Instance pendant l'opération	Hôte source	Choix de la cible	Condition principale	Usage
Migration à chaud	En marche ; pause de quelques instants en fin de transfert	En service	Scheduler ou hôte imposé	Processeurs compatibles ; disque partagé, ou copie des disques	Vider un hôte avant maintenance
Migration à froid	Arrêtée, puis redémarrée	En service	Scheduler	Accès entre hôtes pour copier le disque local	Déplacement sans exigence de continuité
Redimensionnement	Arrêtée, puis redémarrée	En service	Scheduler (souvent un autre hôte)	Nouveau gabarit ; confirmation ou retour arrière	Changer de taille
Évacuation	Déjà arrêtée par la panne	Hors service	Scheduler ou hôte imposé	Hôte déclaré « down » ; disque partagé ou sur volume pour garder les données	Reprise après panne d'hôte
Mise à l'écart (shelve)	Arrêtée, disque sauvegardé	Libéré	Au retour, par le scheduler	—	Libérer des ressources sans perdre l'instance

```
openstack server migrate --live-migration web-01  # à chaud
openstack server migrate web-01                   # à froid
openstack server resize --flavor m1.large web-01
openstack server resize confirm web-01          # ou revert
openstack server evacuate web-01
openstack server shelve web-01
```

Migration à chaud, échange par échange : slide 49.

Évacuer n'est pas migrer : l'évacuation suppose l'hôte mort. Lancée sur un hôte encore vivant mais injoignable, elle démarre un second exemplaire de l'instance sur le même disque — d'où l'intérêt d'isoler l'hôte (coupure électrique commandée) avant d'évacuer.

Suivre un déplacement : `openstack server migration list`, et l'état `VERIFY_RESIZE` qui attend `resize confirm` ou `resize revert`.

Interactions : ce que Nova demande, et à quel moment

	Validation (nova-api)	Ordonnancement	Construction sur l'hôte	Finalisation
Keystone	valide le jeton de l'utilisateur			
Glance	**l'image existe-t-elle ?** propriétés, taille minimale		**télécharge ou clone l'image** vérifie la somme de contrôle	
Placement		**hôtes candidats,** puis réservation sur l'hôte retenu	**en cas d'échec : libération,** nouvel essai sur un suppléant	
Neutron	**réseaux et ports demandés ?** quota de ports		**crée ou complète les ports,** désigne l'hôte de liaison	**événement « port prêt »** attendu avant de démarrer
Cinder	**volumes demandés disponibles ?**		**crée les volumes, obtient** les informations de connexion	
État vu par l'API	**BUILD** task_state : scheduling	**BUILD** task_state : scheduling	**BUILD** networking→block_device_mapping→spawning	**ACTIVE** task_state vide

Commandes et diagnostic

```
# État complet d'une instance (administrateur)
openstack server show web-01 -c status -c OS-EXT-STS:vm_state \
    -c OS-EXT-STS:task_state -c OS-EXT-SRV-ATTR:host -c fault

# Historique des opérations et de leurs étapes
openstack server event list web-01
openstack server event show web-01 req-5f1c2b7e-…

# Services, hyperviseurs, déplacements en cours
openstack compute service list
openstack hypervisor list --long
openstack server migration list

# Cellules et rattachement des hôtes
nova-manage cell_v2 list_cells
nova-manage cell_v2 discover_hosts --verbose

# Contrôles avant montée de version
nova-status upgrade check

# Sur l'hyperviseur : ce que voit libvirt
virsh list --all ; virsh dumpxml instance-0000002a
```

Bloquée à l'étape…	Chercher du côté de…
`scheduling` **, puis ERROR « No valid host »**	Placement, filtres du scheduler
`scheduling` **, sans fin**	Bus de messages, nova-conductor, nova-scheduler
`networking`	Neutron : création ou liaison du port
`block_device_mapping`	Cinder : création ou connexion du volume
`spawning`	Glance (image), libvirt, événement « port prêt » de Neutron
ACTIVE mais injoignable	Groupes de sécurité, DHCP, métadonnées ; plus Nova

- Le champ **fault** donne le message et, pour un administrateur, la trace de l'erreur.
- Les **événements d'instance** listent chaque action, son identifiant de requête et l'étape qui a échoué : le meilleur point d'entrée dans les journaux.
- Le nom libvirt d'une instance (`instance-0000002a`) figure dans `OS-EXT-SRV-ATTR:instance_name` .

04

Cinder — le stockage bloc

Cinder fournit des disques persistants, indépendants des instances. Il pilote des backends très différents derrière une même API, et s'efface dès que le disque est connecté.

- Rôle et notions
- Types de volumes et ordonnancement
- Créer un volume depuis une image
- Architecture multi-backend
- Les états d'un volume
- Instantanés, sauvegardes, interactions

Rôle et notions

Rôle : créer et gérer des **volumes** — des disques virtuels dont la durée de vie ne dépend d'aucune instance — et fournir à l'hyperviseur de quoi s'y connecter.

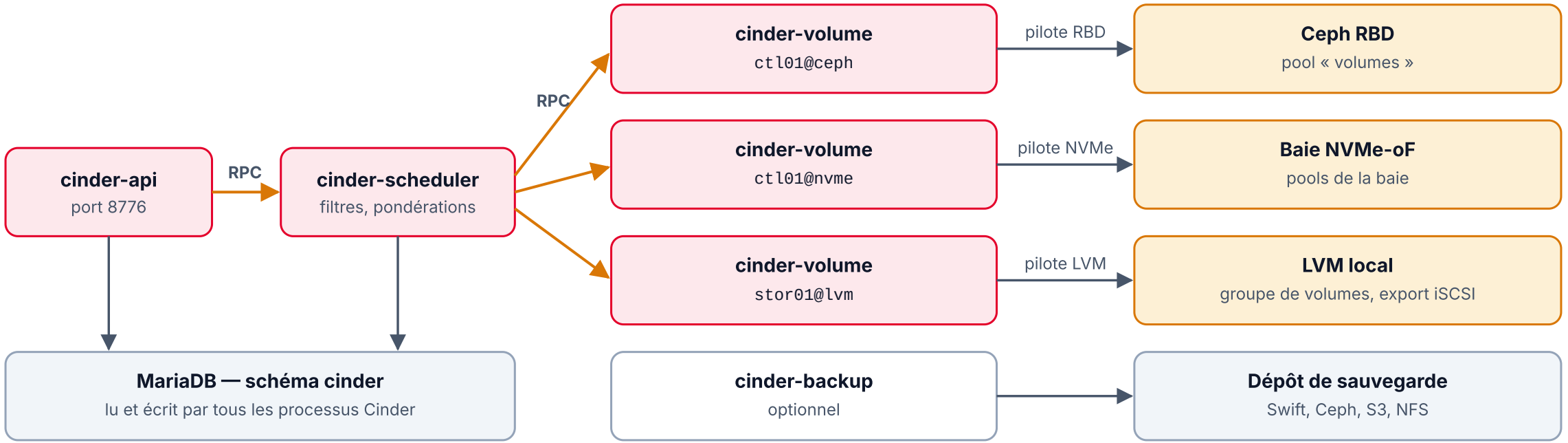
- **Ce qu'il fait :** choisir un backend, y créer le volume, en gérer les instantanés, sauvegardes, clones et extensions, autoriser les connexions.
- **Ce qu'il ne fait pas :** transporter les données. Une fois le volume connecté, l'hyperviseur dialogue directement avec le backend.
- **Conséquence :** un arrêt de Cinder n'interrompt aucune E/S en cours ; il empêche seulement de créer, attacher ou détacher.

Processus	Fonction
cinder-api	API REST ; valide, vérifie quotas, enregistre
cinder-scheduler	Choisit le backend et le pool
cinder-volume	Un par backend ; pilote le stockage
cinder-backup	Copie les volumes vers un dépôt distinct (optionnel)

Notion	Définition
Volume	Disque bloc persistant, d'une taille en Go, appartenant à un projet
Type de volume	Classe de service choisie par l'utilisateur : backend, performances, chiffrement
Instantané	État figé d'un volume, sur le même backend ; base d'un nouveau volume
Sauvegarde	Copie hors du backend, complète ou incrémentale
Attachement	Lien entre un volume et une instance, avec ses informations de connexion
Volume amorçable	Créé depuis une image ; peut servir de disque système
Spécification de qualité de service	Limites de débit et d'opérations, appliquées par le backend ou l'hyperviseur

Différence avec le disque éphémère de Nova : un volume survit à l'instance, se détache, se rattache ailleurs, se sauvegarde. C'est le support des données qui comptent.

Architecture : un service de volume par backend




```
# /etc/cinder/cinder.conf
[DEFAULT]
enabled_backends = ceph, nvme
[ceph]
volume_driver = cinder.volume.drivers.rbd.RBDDriver
volume_backend_name = ceph
rbd_pool = volumes
rbd_user = cinder
```

- Chaque backend est une **section** du fichier et donne naissance à un service de volume nommé `hôte@backend`.
- Un backend peut exposer plusieurs **pools** : la désignation complète est `hôte@backend#pool`.
- Backend distant (Ceph, baie) : cinder-volume tourne sur les contrôleurs. Backend local (LVM) : il tourne sur le nœud qui porte les disques.

Types de volumes et choix du backend



Chaque cinder-volume publie périodiquement les capacités de ses pools : espace total et libre, fonctions prises en charge, nom de backend.

```
# Créer un type et le lier à un backend
openstack volume type create rapide
openstack volume type set rapide \
    --property volume_backend_name=nvme

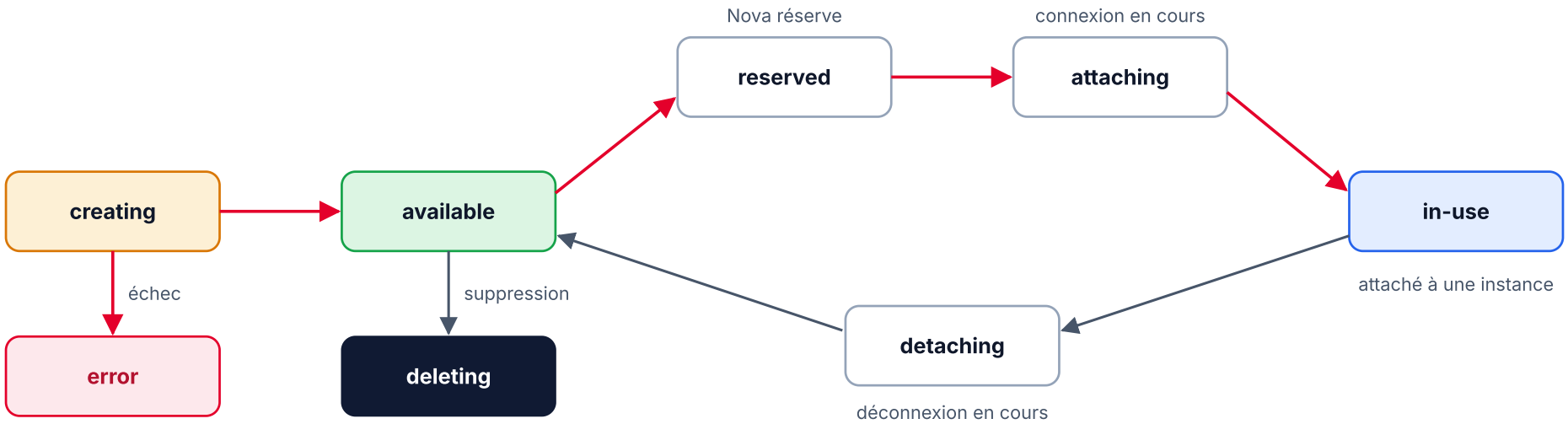
# Limiter les performances
openstack volume qos create bridee --consumer front-end \
    --property total_iops_sec=2000
openstack volume qos associate bridee rapide

# Voir les pools et ce qu'ils annoncent
openstack volume backend pool list --long
```

Étape	Composant par défaut
Filtres	`AvailabilityZoneFilter`, `CapacityFilter`, `CapabilitiesFilter`
Pondération	`CapacityWeigher` : privilégie l'espace libre

- Le **type** est le seul levier de l'utilisateur : il ne voit ni les backends ni les pools.
- Les propriétés du type (*extra specs*) sont comparées aux capacités annoncées : nom de backend, provisionnement fin, réplication, chiffrement.
- Qualité de service `front-end` : appliquée par l'hyperviseur (libvirt) ; `back-end` : par le stockage.
- Un type peut être **privé** : réservé à certains projets.

Les états d'un volume

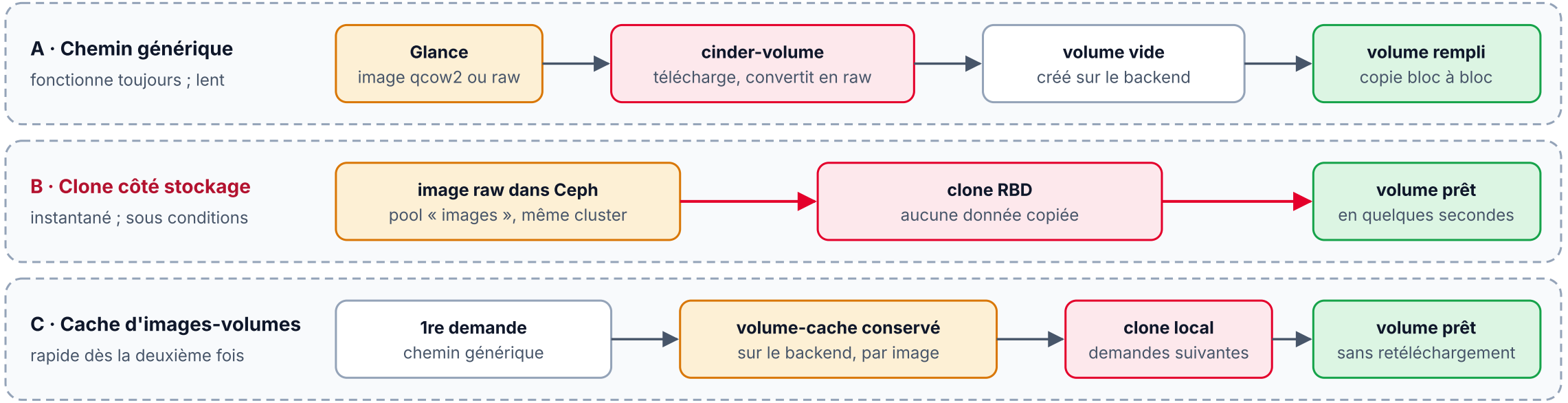


État transitoire	Opération en cours
extending	Agrandissement ; possible volume attaché si le backend le permet
backing-up, restoring-backup	Sauvegarde ou restauration
uploading, downloading	Export vers une image Glance ; remplissage depuis une image
retyping, migrating	Changement de type ou de backend
error_extending, error_deleting…	L'opération a échoué ; le volume reste dans cet état

- Un volume ne s'attache que s'il est **available** (ou déjà *in-use* avec l'attachement multiple).
- Un volume bloqué dans un état transitoire refuse toute action : c'est le symptôme d'un ordre RPC perdu ou d'un cinder-volume arrêté.
- Remise en état par l'administrateur : `openstack volume set --state available` — après avoir vérifié la réalité côté backend et côté Nova.

> Un volume a aussi un **état d'attachement** et un **état de migration** distincts : les trois doivent être cohérents avant de forcer quoi que ce soit.

Créer un volume depuis une image : trois chemins



Chemin	Conditions	Coût
A · Générique	Aucune	Temps et espace temporaire, selon la taille
B · Clone	Image raw, même cluster Ceph, emplacement exposé	Quasi nul
C · Cache	`image_volume_cache_enabled` sur le backend	Un volume caché par image

- Le chemin A exige de l'**espace disque temporaire** là où tourne cinder-volume : une image de 40 Go qui échoue « sans raison » a souvent rempli ce répertoire.
- Le chemin B crée une **dépendance** entre le volume et l'image : tant que le clone existe, l'image ne peut pas être supprimée de Ceph.
- Dans tous les cas, le volume reçoit les métadonnées de l'image (`volume_image_metadata`) : Nova les relira au démarrage.

Instantanés, sauvegardes, clones et déplacements

Opération	Ce qu'elle produit	Où	Volume attaché ?	Usage
Instantané	État figé du volume, à un instant	Même backend que le volume	Oui, avec `--force` ; cohérence à assurer côté invité	Point de retour rapide ; source d'un nouveau volume
Clone	Nouveau volume, copie d'un volume existant	Même backend	Oui	Dupliquer un environnement
Sauvegarde	Copie complète ou incrémentale	Dépôt distinct : Swift, Ceph, S3, NFS	Oui, avec `--force`	Protection contre la perte du backend
Changement de type (retype)	Même volume, autre classe de service	Sur place, ou vers un autre backend	Oui, si le backend le permet	Passer de « standard » à « rapide »
Migration	Même volume, autre backend ou pool	D'un backend à l'autre	Oui : Nova bascule le disque à chaud	Vider un backend avant son retrait
Transfert	Même volume, autre projet	Sur place	Non	Céder un volume à une autre équipe

Instantané n'est pas sauvegarde : l'instantané vit sur le même stockage que le volume et disparaît avec lui. Seule la sauvegarde sort les données du backend.

Dépendances : sur beaucoup de backends, un volume ne peut pas être supprimé tant qu'il a des instantanés, et un instantané tant que des volumes en sont issus. Vérifier avant de conclure à un blocage.

Interactions, commandes et diagnostic

Avec	Échange
Nova	Nova réserve, fournit la description de l'hôte, reçoit les informations de connexion ; Cinder prévient Nova d'une extension
Glance	Volume depuis une image ; image depuis un volume ; magasin d'images adossé à Cinder
Keystone	Jetons, et jeton de service exigé pour les actions réservées à Nova
Barbican	Clés des volumes chiffrés
Swift, Ceph, S3	Dépôt des sauvegardes
Backend	Création, export, droits d'accès — par le pilote

Qui établit la connexion : os-brick, sur l'hyperviseur, d'après la réponse de Cinder. Un échec de connexion se lit dans le journal de **nova-compute**.

Symptôme	Piste
Volume en « error » dès la création	« No valid backend » : type, capacité, zone
Figé en « creating », « attaching »	cinder-volume arrêté, ordre RPC perdu
Attachement refusé (403)	Jeton de service absent côté Nova

```
# Services et backends vivants
openstack volume service list
openstack volume backend pool list --long

# Où vit ce volume, dans quel état, attaché à quoi
openstack volume show data-01 -c status -c type \
    -c os-vol-host-attr:host -c attachments

# Les attachements, vus de Cinder
openstack volume attachment list --os-volume-api-version 3.27

# Pourquoi une création a échoué
openstack volume message list

# Cycle courant
openstack volume create --size 50 --type rapide data-01
openstack server add volume web-01 data-01
openstack volume snapshot create --volume data-01 --force s1
openstack volume backup create --force data-01

# Côté Ceph : le volume existe-t-il vraiment ?
rbd -p volumes ls | grep <id-du-volume>
```

`volume message list` donne en clair la raison d'un échec asynchrone, sans avoir à lire les journaux.

05

Neutron — le réseau

Neutron donne à chaque projet ses réseaux, ses routeurs et ses règles de filtrage. Il décrit le réseau voulu ; un backend — OVN dans la configuration de référence — le réalise sur chaque hyperviseur.

- Rôle et modèle de ressources
- Du modèle Neutron au modèle OVN
- Filtrage, chemins du trafic, services intégrés
- Architecture : ML2 et ses pilotes
- La liaison de port
- Interactions et diagnostic

5 · NEUTRON

Rôle et modèle de ressources

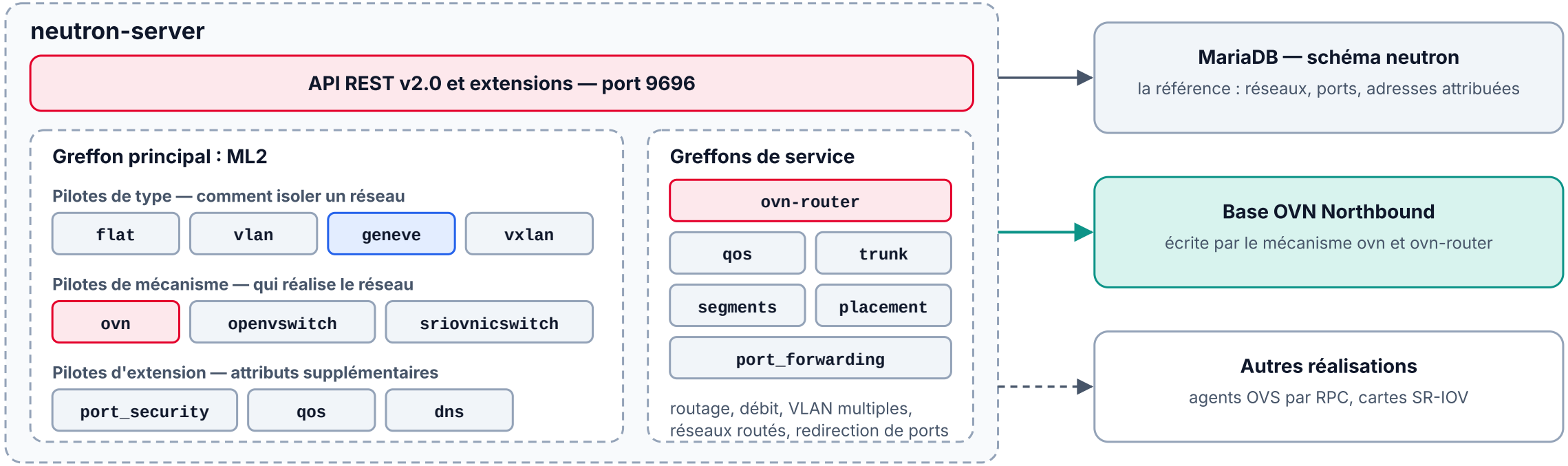
Rôle : offrir le réseau « à la demande ». Chaque projet compose sa topologie par API, isolée de celle des autres, sans intervention de l'équipe réseau.

- **Ce qu'il fait :** tenir le modèle (réseaux, sous-réseaux, ports, routeurs), attribuer les adresses, traduire ce modèle pour le backend, signaler à Nova qu'un port est prêt.
- **Ce qu'il ne fait pas :** commuter les paquets. Le trafic passe par Open vSwitch sur chaque hôte, jamais par neutron-server.

Le port est l'objet central : c'est lui qui relie une instance (côté Nova) à un réseau (côté Neutron). Adresse MAC, adresses IP, groupes de sécurité, hôte de rattachement : tout s'y trouve.

Ressource	Rôle	Créée par
Réseau	Domaine de diffusion isolé	Projet, ou administrateur (fournisseur)
Sous-réseau	Plage d'adresses, passerelle, DHCP, DNS	Projet
Port	Point de raccordement	Projet, ou Nova pour son compte
Routeur	Relie des sous-réseaux, sort vers l'extérieur	Projet
IP flottante	Adresse externe associée à un port	Projet
Groupe de sécurité	Règles de filtrage appliquées aux ports	Projet
Réseau fournisseur	Adossé à un VLAN ou réseau physique	Administrateur

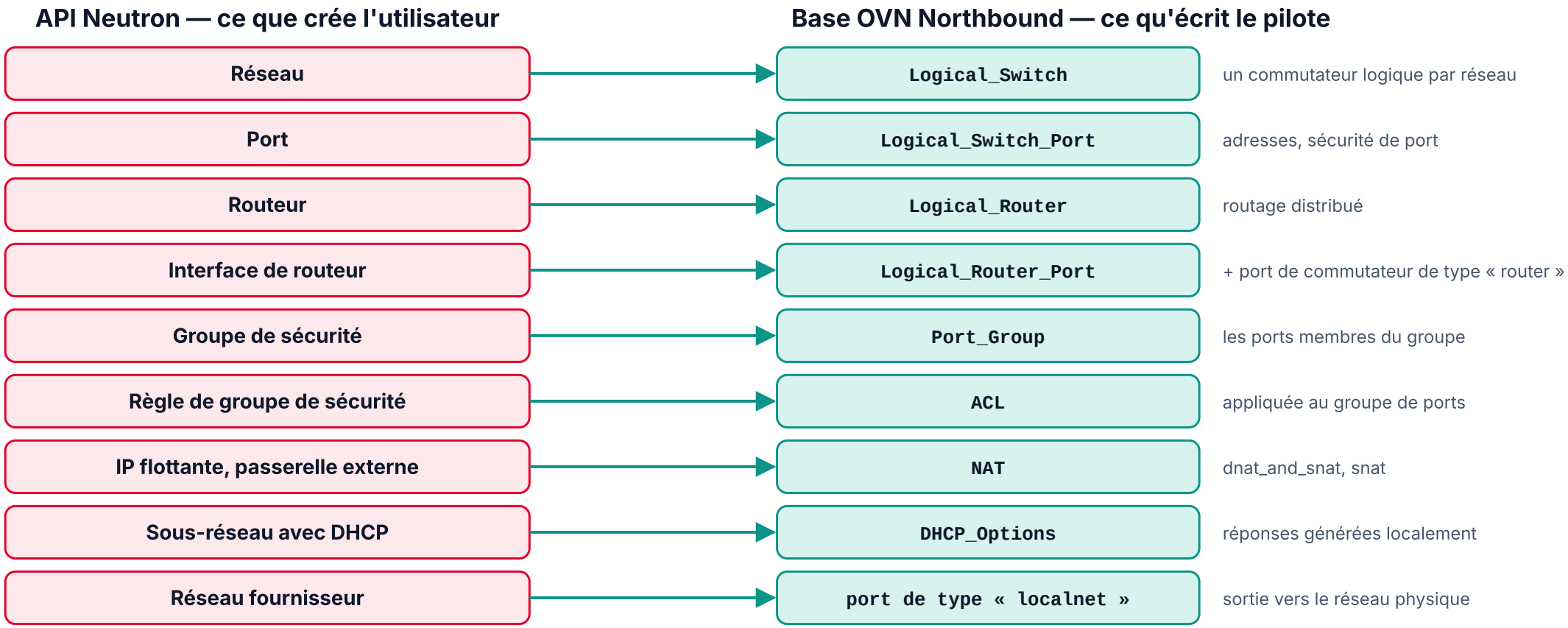
Architecture : neutron-server, ML2 et ses pilotes




```
# /etc/neutron/plugins/ml2/ml2_conf.ini
[ml2]
type_drivers = flat,vlan,geneve
tenant_network_types = geneve
mechanism_drivers = ovn
extension_drivers = port_security,qos,dns
[ml2_type_vlan]
network_vlan_ranges = physnet1:100:199
```

- **ML2 sépare deux questions :** le *type* (comment un réseau est isolé) et le *mécanisme* (qui le met en œuvre). Plusieurs mécanismes peuvent cohabiter.
- `tenant_network_types` : type attribué aux réseaux que créent les projets — ici des tunnels Geneve.
- `network_vlan_ranges` : VLAN utilisables pour les réseaux fournisseurs, par réseau physique nommé.

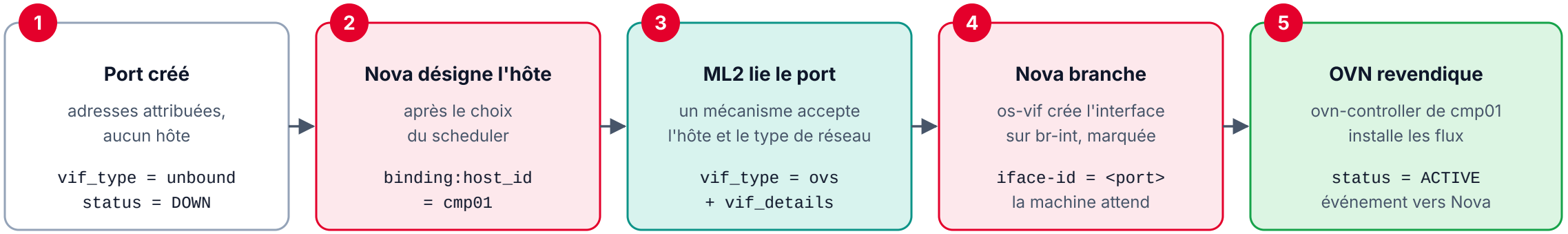
Du modèle Neutron au modèle OVN



Lecture : le pilote `ovn` traduit chaque objet de l'API en une ou plusieurs lignes de la base Northbound. `ovn-northd` en déduit ensuite les flux logiques, que chaque ovn-controller convertit en règles Open vSwitch.

En cas d'écart entre les deux bases (panne pendant une écriture, intervention manuelle), l'outil `neutron-ovn-db-sync-util` compare et, en mode réparation, réaligne OVN sur Neutron.

La liaison de port : relier un port à un hôte

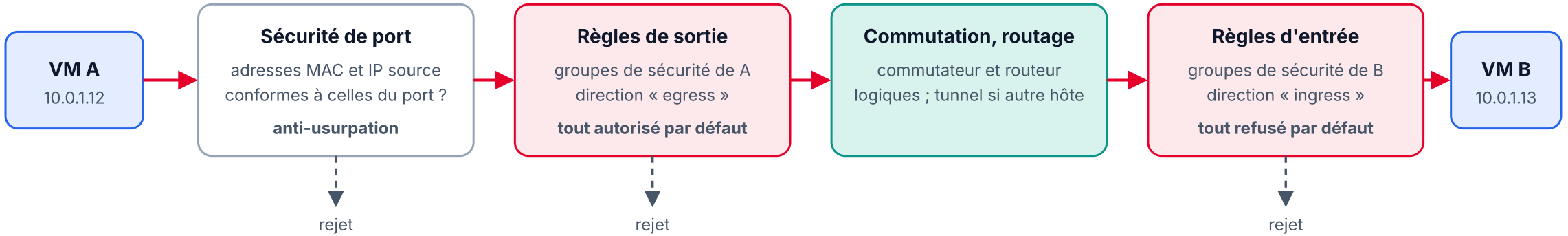


Attribut du port	Sens
`binding:host_id`	Hôte sur lequel le port doit vivre ; renseigné par Nova
`binding:vnic_type`	Nature de l'interface : `normal` (virtuelle), `direct` (SR-IOV), `baremetal`
`binding:vif_type`	Résultat : `ovs`, `unbound`, ou `binding_failed`
`binding:vif_details`	Ce que Nova doit savoir pour brancher : type de pont, filtrage
`binding:profile`	Informations fournies par Nova : adresse PCI, données de migration
`device_owner`, `device_id`	Qui utilise le port : instance, routeur, DHCP, IP flottante

- `binding_failed` : aucun mécanisme n'a pu lier le port à cet hôte — hôte inconnu d'OVN, réseau physique non déclaré sur l'hôte (`bridge-mappings`), type de réseau non pris en charge.
- Le lien entre l'interface locale et le port logique se fait par l'identifiant du port, posé sur l'interface Open vSwitch.
- **Migration à chaud :** un port peut porter deux liaisons, source et cible ; la seconde est activée au moment de la bascule.

> Un port DOWN sur une instance ACTIVE mérite toujours un regard : `openstack port show <id> -c binding_vif_type -c binding_host_id -c status` .

Groupes de sécurité : le filtrage au plus près du port



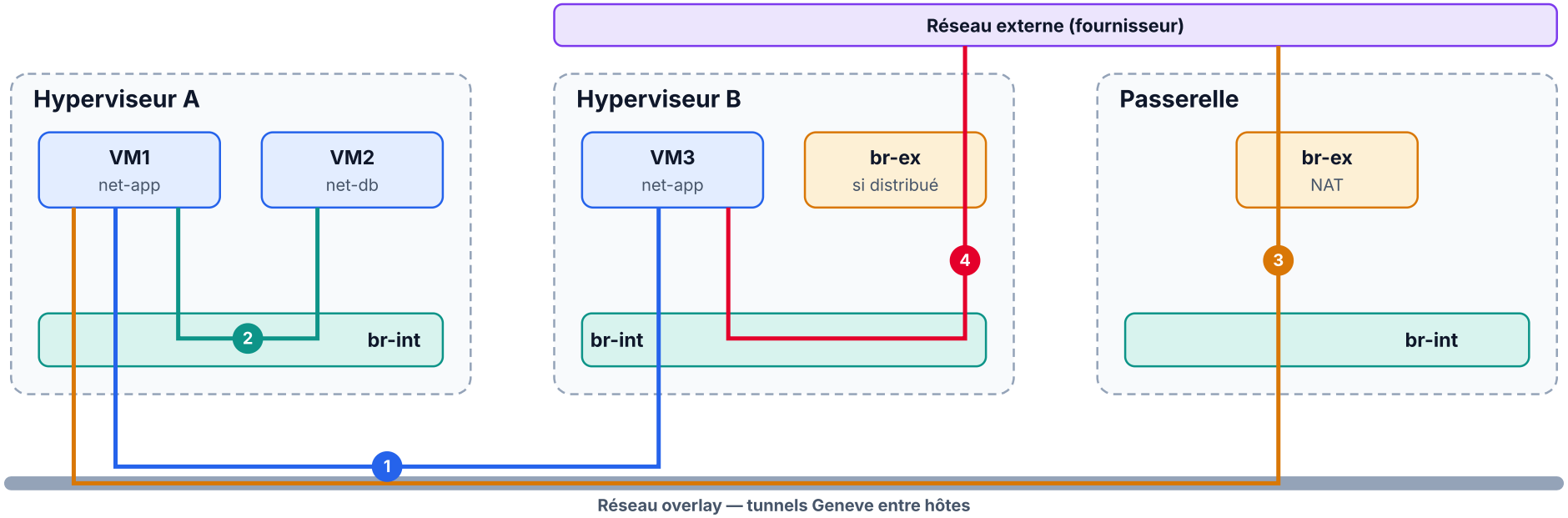
Suivi de connexion : une fois un échange autorisé, les paquets de réponse passent sans règle dédiée.

- Les règles s'appliquent **sur l'hyperviseur**, à l'entrée et à la sortie de chaque port — y compris entre deux instances du même réseau.
- Avec OVN : un groupe de sécurité devient un **groupe de ports**, chaque règle une **ACL** ; le tout est exécuté par les flux Open vSwitch, avec suivi de connexion du noyau.
- Une règle peut désigner comme source un **autre groupe** : « autoriser 5432 depuis le groupe web ». L'appartenance est suivie automatiquement.
- Le groupe `default` de chaque projet autorise le trafic entre ses propres membres et toute sortie.

Réglage du port	Effet
`port_security_enabled = false`	Ni anti-usurpation ni groupes : pour un routeur ou pare-feu virtuel
`allowed_address_pairs`	Autorise des adresses supplémentaires : adresse virtuelle partagée (VRRP)

```
openstack security group rule create web \
    --ingress --protocol tcp --dst-port 443 \
    --remote-ip 0.0.0.0/0
# Côté OVN : les ACL du groupe
ovn-nbctl acl-list pg_<id_du_groupe>
```

Les chemins du trafic



	Cas	Chemin	Où se fait le traitement
1	Même réseau, hôtes différents	VM1 → br-int A → tunnel → br-int B → VM3	Commutation logique sur A ; aucun équipement central
2	Deux réseaux reliés par un routeur	VM1 → br-int A → VM2	Routage **distribué** : exécuté sur l'hyperviseur source
3	Sortie sans IP flottante	VM1 → tunnel → passerelle → br-ex → externe	Traduction d'adresse source (SNAT) **centralisée** sur la passerelle
4	IP flottante, mode distribué	VM3 → br-int B → br-ex B → externe	NAT sur l'hyperviseur lui-même, s'il est raccordé au réseau externe

> Le mode distribué des IP flottantes (`enable_distributed_floating_ip`) suppose que **chaque hyperviseur** ait une patte sur le réseau externe. Sinon, les IP flottantes passent elles aussi par la passerelle, comme le cas 3.

5 · NEUTRON

Adresses, DHCP, DNS, métadonnées : les services intégrés

Service	Qui le rend	Comment	Point d'attention
Attribution d'adresses (IPAM)	neutron-server	À la création du port, une adresse est prise dans la plage du sous-réseau et enregistrée en base	L'adresse appartient au **port**, pas à l'instance : elle survit à un redémarrage, pas à la suppression du port
DHCP	OVN, sur chaque hyperviseur	ovn-controller répond lui-même aux requêtes, d'après les options inscrites en base	Aucun serveur ni agent DHCP ; les options du sous-réseau (DNS, routes, MTU) y sont reprises
DNS interne	OVN	Les noms des ports d'un réseau sont résolus localement	Requiert le pilote d'extension `dns` ; publication externe via Designate
Métadonnées	ovn-metadata-agent, sur chaque hyperviseur	Intercepte `169.254.169.254`, ajoute l'identité de l'instance, relaie vers l'API de Nova	Un espace de noms et un relais par réseau présent sur l'hôte
Routage, NAT	OVN	Routeurs logiques exécutés par les flux Open vSwitch	Pas d'espace de noms de routeur ; SNAT sur les passerelles
Répartition de charge	Octavia, avec le fournisseur OVN ou des amphores	Service distinct qui s'appuie sur les ports Neutron	Hors de Neutron lui-même

Tout est local à l'hyperviseur : avec OVN, une instance obtient son adresse, résout un nom et lit ses métadonnées sans qu'aucun paquet ne quitte son hôte.

Vérifier depuis l'instance : bail DHCP reçu ? `curl http://169.254.169.254/openstack/latest/meta_data.json` répond ? Deux tests qui valident d'un coup le port, le filtrage et les relais.

Interactions, commandes et diagnostic

Avec	Échange
Nova	Nova crée ou complète les ports et désigne l'hôte ; Neutron renvoie les événements « port prêt », « réseau modifié »
Placement	Neutron déclare la bande passante des interfaces et les segments de réseaux routés ; Nova en tient compte au placement
Keystone	Jetons ; Neutron a son propre compte pour appeler Nova
Designate	Publication des noms des ports et des IP flottantes
Octavia	Ports, adresses virtuelles et groupes de sécurité des répartiteurs
Ironic	Ports des serveurs physiques, configuration des commutateurs

Symptôme	Piste
Instance sans adresse	Port DOWN, liaison en échec, ovn-controller arrêté
Ping impossible entre instances	Groupe de sécurité ; MTU ; tunnel entre hôtes
IP flottante injoignable	Passerelle, `bridge-mappings`, règle d'entrée
Métadonnées muettes	ovn-metadata-agent sur l'hôte ; secret partagé avec Nova

```
# Le port d'une instance : état et liaison
openstack port list --server web-01
openstack port show <id> -c status -c binding_host_id \
    -c binding_vif_type -c fixed_ips -c security_group_ids

# Les hôtes connus du réseau
openstack network agent list

# Vue OVN : topologie logique, puis hôtes et ports liés
ovn-nbctl show
ovn-sbctl show
ovn-sbctl find Port_Binding logical_port=<id_du_port>

# Simuler le trajet d'un paquet dans le réseau logique
ovn-trace neutron-<id_reseau> 'inport == "<port_A>" &&
    eth.src == fa:16:3e:aa:bb:01 && ip4.src == 10.0.1.12 &&
    ip4.dst == 10.0.1.13 && tcp.dst == 443'

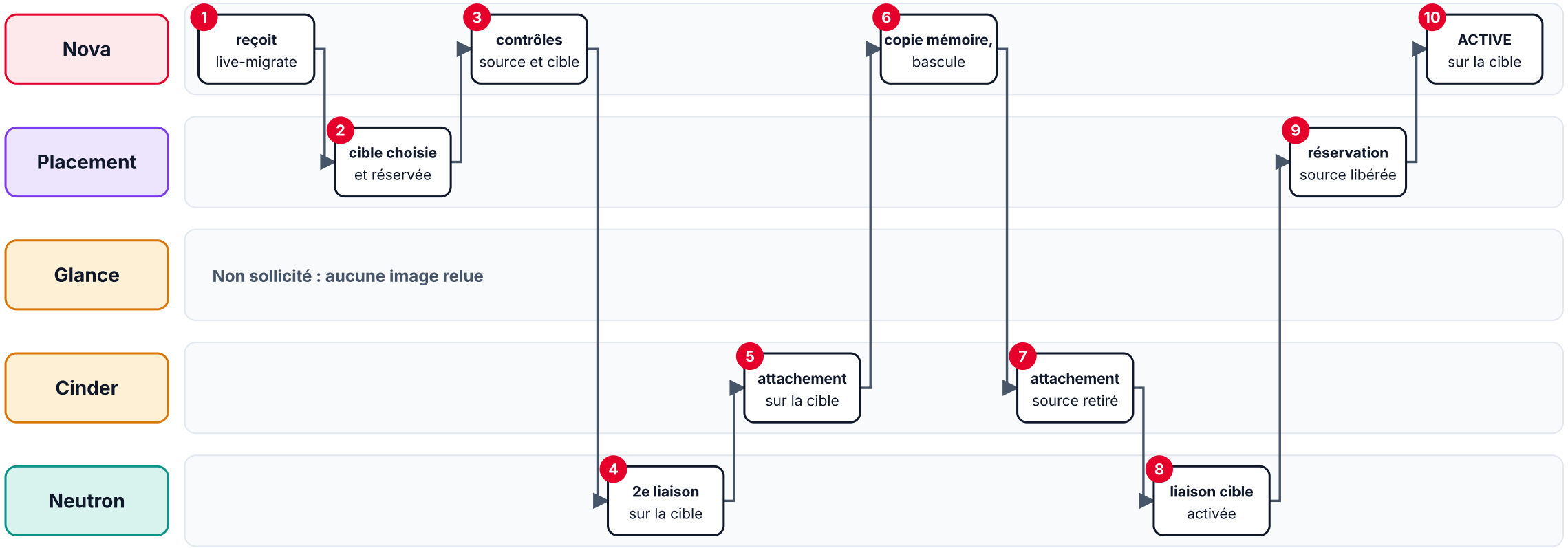
# Sur l'hyperviseur : l'interface est-elle sur br-int ?
ovs-vsctl list-ports br-int

# Réaligner OVN sur Neutron
neutron-ovn-db-sync-util --ovn-neutron_sync_mode repair
```

> `ovn-trace` dit quelle règle laisse passer ou rejette un paquet, sans rien émettre sur le réseau.

SYNTHÈSE

Les services dans une même opération : la migration à chaud



Scénario : migrer à chaud une instance démarrée sur un volume Ceph. **Placement** garde la réservation des deux côtés jusqu'à la fin (celle de la source passe au nom de la migration) ; **Neutron** et **Cinder** préparent la cible avant la bascule, puis la source est nettoyée. **Glance** n'intervient pas : le disque n'est pas recréé.

SYNTHÈSE

À retenir

	Glance	Placement	Nova	Cinder	Neutron
Rôle	Catalogue et distribution des images	Comptabilité des ressources	Cycle de vie des instances	Disques persistants	Réseaux à la demande
Objet suivi	Image : queued → active	Allocation	Instance : BUILD → ACTIVE	Volume : available ↔ in-use	Port : DOWN → ACTIVE
Appelle	Son magasin	Personne	Les quatre autres	Glance, Nova	Nova, Placement
S'il s'arrête	Plus de création depuis une image non mise en cache	Plus de création ni de déplacement	Plus d'opération sur les instances	Plus de création ni d'attachement	Plus de modification du réseau
Ce qui continue	Les instances tournent, leurs disques restent connectés, leur trafic circule : aucun de ces services n'est sur le chemin des données				
Première commande	`image show`	`allocation candidate list`	`server event list`	`volume message list`	`port show`

Un patron commun

API qui valide, composant qui choisit, exécutant qui agit par un pilote. Le reconnaître dans un service, c'est savoir lire les quatre autres.

Des machines à états

Chaque ressource avance d'état en état. Un état transitoire qui dure désigne l'étape — donc le service — où chercher.

Nova assemble

Il emprunte l'image, la place, le port et le volume. Une instance qui ne démarre pas est le plus souvent l'échec d'un de ces emprunts.